KABC Benchmark Results: Experiment 1 (Gaussian Mixtures) and Experiment 3 (MovieLens-1M)

Kernel-based Active Clustering of Bandit Arms
Thuot, Vogt, Ghoshdastidar, Verzelen (arXiv:2601.07535)

September 12, 2026

This report summarizes the experimental setup, baseline algorithms, results, and conclusions for Experiments 1 and 3 of the `kernel-active-bandit` benchmark suite. Full design rationale lives in `docs/experiments.md`; this document summarizes what was actually run and what the real (non-smoke-test) results show.

Run provenance. `results/exp1/results.parquet` holds 10,400 rows (13 grid points × 2 budget multipliers × 4 algorithms × 100 reps, ≈24.4 hours wall-clock). `results/exp3/results.parquet` holds 550 rows (2 grid points, 100 reps per algorithm except TaS-FW's 50, ≈66 minutes wall-clock). Both were generated by `scripts/run_experiment.py` using calibrated threshold constants (Section 3). Figures were regenerated from these files by `notebooks/01_gaussian_mixtures.ipynb` and `notebooks/03_movielens.ipynb`.

Contents

1 Experimental Setup **2**
1.1 Experiment 1 — Gaussian mixtures: competitiveness in the parametric regime . . . 2
1.2 Experiment 3 — MovieLens: real rating distributions . . . 2

2 Baseline Algorithms **3**
2.1 KABC and KABC-SG (this paper's own methods) . . . 3
2.2 ACB — Active Clustering with Bandit feedback . . . 4
2.3 TaS-FW — Track-and-Stop with Frank-Wolfe . . . 4
2.4 Unif-KKM — uniform sampling + kernel k-means (folklore baseline) . . . 4

3 Calibration **4**

4 Results — Experiment 1 **5**
4.1 Guaranteed budget (budget_multiplier = 1.0): does the theory hold? . . . 5
4.2 Restricted budget (budget_multiplier = 0.25): graceful degradation? . . . 6

5 Results — Experiment 3 **9**

6 Inferences and Conclusions **9**

1 Experimental Setup

1.1 Experiment 1 — Gaussian mixtures: competitiveness in the parametric regime

Claim tested. In the mean-separated Gaussian setting (home turf of mean-based methods like ACB), KABC's budget is within a modest factor of ACB's; budget scales as $s_*^{-2}\log(N/\delta)$ (the paper's Theorem 3.1).

Instance family. `GaussianMixtureEnv`: N arms in $K = 3$ balanced clusters, arm i in cluster k distributed as $\mathcal{N}(\mu_k, \sigma^2 I_d)$, $\sigma = 1$. Cluster means $\mu_k = \frac{\Delta}{\sqrt{2}}(e_{2k-1} + e_{2k})$, giving each pair of cluster means an actual Euclidean separation of $\Delta\sqrt{2}$.

Grid. One knob swept at a time, others held at the base config N=60, K=3, d=10, Δ=3:

- $\Delta \in \{1, 1.5, 2, 3, 4, 6\}$ (6 points — the headline sweep)
- $d \in \{6, 10, 50, 100\}$ (4 points; $d = 2$ excluded, construction requires $d \geq 2K = 6$)
- $N \in \{30, 60, 120\}$ (3 points)

13 distinct grid points total (the base config is each sweep's own midpoint, so it is run identically under all three sweep tags — a documented, accepted redundancy).

Budget regimes. Every grid point above is run under two multipliers:

- budget_multiplier = 1.0: the δ-PAC *guaranteed budget*, budget_cap = $200\, N\, s_*^{-2}\log(N/\delta)$, s_* computed numerically per grid point.
- budget_multiplier = 0.25: a new *restricted budget* pass, same grid, budget cut to a quarter. Motivation: real deployments of active clustering often cannot afford the full guaranteed-budget amount; this measures graceful degradation under a hard 4× cut, complementary to (not a replacement for) the guaranteed-budget story.

This gives $13 \times 2 = 26$ total grid points.

Protocol. $\delta = 0.05$ throughout. 100 Monte Carlo repetitions per (grid point, algorithm). seed = $1000 \cdot$ exp_id + rep. Pilot pulls ($n_{\text{pilot}} = 5$/arm, median-heuristic bandwidth estimation) count toward the reported τ.

Algorithms. `kabc`, `kabc-sg`, `acb`, `unif-kkm` (Section 2). `tas-fw` is *not* run here — its finite-alphabet assumption does not hold for continuous Gaussian instances.

1.2 Experiment 3 — MovieLens: real rating distributions

Claim tested. The "same mean, different shape" phenomenon that mean-based methods cannot see (demonstrated on synthetic data in Experiment 2) also occurs in real data (polarizing vs. consensus movies), and KABC exploits it where mean-based methods fail.

Data. MovieLens-1M (`data/ml-1m/`), filtered to movies with ≥ 1000 ratings.

Prototype selection (deterministic, scripted — `MovieLensEnv`):

1. For each retained movie, compute its empirical rating distribution on $\{1, \ldots, 5\}$ and pairwise MMD^2 (RBF kernel, median-heuristic bandwidth).
2. Find the pair maximizing MMD^2 subject to $|\text{mean gap}| \leq 0.05$ — the "polarizing vs. consensus" same-mean pair.
3. Add $K - 2$ further prototypes greedily maximizing minimum pairwise MMD^2.
4. $K = 4$ prototypes total, 10 arms/prototype ($N = 40$, main instance). Pulling an arm draws one rating with replacement from its prototype's pool.

On the real data, the same-mean pair selected was *Clear and Present Danger* (1994, mean rating 3.73) vs. *South Park: Bigger, Longer and Uncut* (1999, mean rating 3.76) — a 3-star-consensus family drama against a bimodal love-it-or-hate-it comedy: essentially the same mean rating, very different distribution shape (Figure 6, left panel).

A separate, much smaller *reduced* instance ($K = 3$ prototypes, 2 arms each, $N = 6$) is built so TaS-FW's finite-alphabet assumption holds natively (MovieLens ratings *are* a finite alphabet, $\{1, \ldots, 5\}$), letting it run as a fair baseline.

Budget. Main instance: fixed budget_cap = 2,000,000 pulls (no s_*-based formula — s_* is not in closed form for real, empirical rating distributions). Reduced instance: budget_cap = 20,000.

Protocol. $\delta = 0.05$. 100 reps/algorithm (50 for TaS-FW). No budget-multiplier sweep for Experiment 3.

Algorithms. Main instance — `kabc`, `kabc-sg`, `acb`, `unif-kkm`. Reduced instance — `tas-fw`, `kabc`.

2 Baseline Algorithms

2.1 KABC and KABC-SG (this paper's own methods)

KABC (Algorithm 2 of [1]) runs a doubling-trick outer loop: at iteration k, pull $n_k = \lceil 2^k \log(8(N^2 - N)/\delta_k) \rceil$ fresh samples per arm ($\delta_k = \delta/(4k^2)$), then build a same-cluster graph over all arm pairs via a pairwise MMD-based test — an edge $\{i, j\}$ is added iff the empirical MMD statistic between arms i and j falls under a threshold. The recovered partition is the graph's connected components; the loop stops the first time the number of connected components equals the true K.

Two threshold variants are compared (identical outer loop, different pairwise test):

`kabc` variance-aware, Wolfer–Alquier-style threshold (Eq. 6 of [1]): sizes each pair's own threshold from that pair's *empirical* RKHS variance, so a noisier pair automatically gets more slack.

`kabc-sg` kernel-uniform sub-Gaussian threshold (the Discussion section's ablation in [1]): uses only the fixed kernel bound (RBF kernel sup = 1), identical for every pair regardless of that pair's own spread — the ablation whose practical fragility Section 6 discusses in detail.

2.2 ACB — Active Clustering with Bandit feedback

Source: [2]. ACB* (this repo's adaptive variant, "θ_* and Δ_* unknown," referred to throughout as ACB) is a *mean*-based active clustering procedure (SRI + ADC subroutines): it estimates each arm's mean directly (no kernel/MMD machinery) and clusters by Euclidean distance between empirical means, adaptively refining which arms to pull based on estimated separation. It is the "home turf" competitor for Experiment 1 (mean-separated Gaussians, exactly ACB's design setting) and the mean-blindness foil for Experiment 3 (same-mean, different-shape data, where a mean-only method has no signal by construction).

2.3 TaS-FW — Track-and-Stop with Frank-Wolfe

Source: [3]. Track-and-Stop based on Frank-Wolfe optimization, for N-ary clustering of arms whose distributions live on a common *finite* alphabet. Only run on Experiment 3's reduced ($N = 6$) instance, where MovieLens ratings on $\{1, \ldots, 5\}$ satisfy the finite-alphabet assumption natively; not run on Experiment 1's continuous Gaussian instances, which would require an off-label discretization approximation outside the algorithm's proven guarantees.

2.4 Unif-KKM — uniform sampling + kernel k-means (folklore baseline)

No paper — a standard, non-adaptive baseline implemented directly per the design doc. Pulls a fixed number of samples per arm (sized to the same budget cap the adaptive methods are given), then runs ordinary kernel k-means on the resulting empirical kernel mean embeddings. Has no confidence-based stopping rule and no δ-PAC guarantee — it always spends its entire allotted budget and always "completes" (stopped = True by construction), which is why it is included purely as a non-adaptive reference point, not as a δ-correct competitor.

3 Calibration

`kabc`, `kabc-sg`, and `acb` each have a theoretically-derived but conservative leading constant in their stopping threshold. Per the calibration protocol (`docs/experiments.md` §0.2), each constant was empirically calibrated once (binary search for the smallest constant keeping empirical error $\leq \delta/2$), then frozen and reused unchanged across every experiment. The frozen values actually used for this report:

Algorithm	c_{cal}	Calibration-set error	Held-out error
`kabc`	0.103516	0.000	0.000
`kabc-sg`	0.121094	0.000	0.000
`acb`	0.658984	0.020	0.010
`tas-fw`	— (no calibratable constant)	—	—

Important caveat. Calibration instances are a *single* fixed Gaussian-mixture instance (`GaussianMixtureEnv`'s construction is deterministic given N, K, d, Δ — there is no way to draw "20 different instances," only 20 independently-seeded noise draws of the one instance). The frozen constants above were validated against that one instance's own real budget regime, *not* against Experiment 1's full, more varied grid. Section 6 shows this matters directly: `kabc-sg`'s and `acb`'s calibration does not fully generalize to every real Experiment 1 grid point. `kabc-sg`'s frozen value (0.121094) was itself a

deliberate, documented override of the raw binary-search answer (0.117578), chosen for margin above a demonstrated instability rather than pure search-set efficiency.

4 Results — Experiment 1

4.1 Guaranteed budget (budget_multiplier = 1.0): does the theory hold?

Aggregated over all 13 grid points, 100 reps each (1,300 runs/algorithm):

Algorithm	Stopped rate	Error rate	Mean ARI
`kabc`	100.0%	0.0%	1.000
`unif-kkm`	100.0%	0.0%	1.000
`acb`	100.0%	10.1%	0.987
`kabc-sg`	65.5%	34.5%	0.870

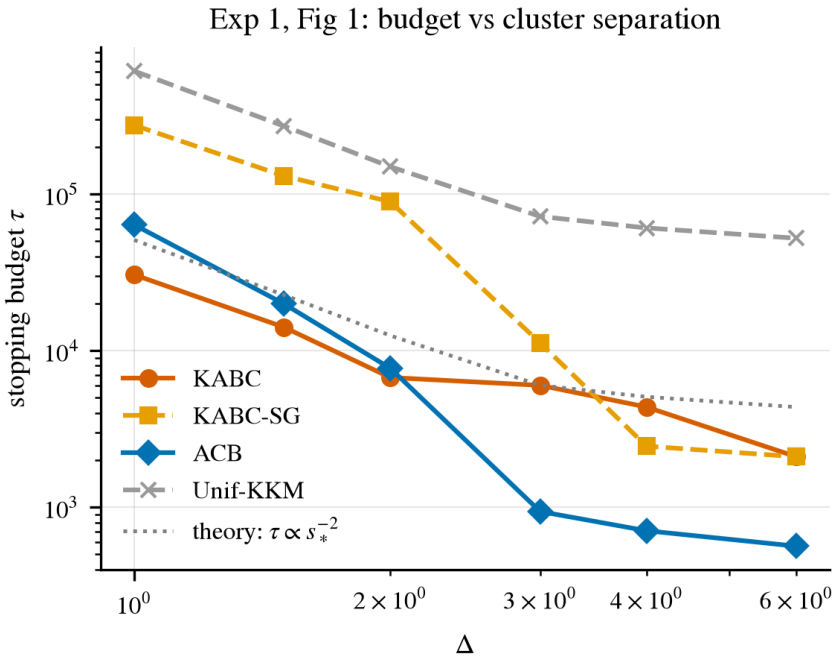
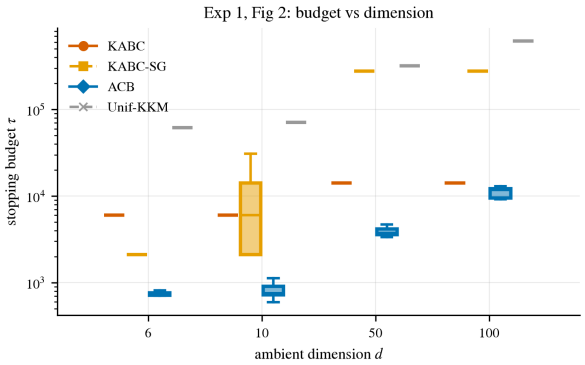


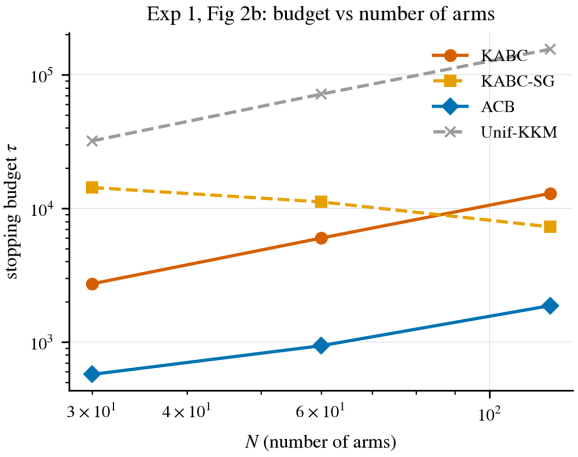
Figure 1: Headline figure: $\log \tau$ vs. $\log \Delta$, all four methods, with the theoretical $\tau \propto s_*^{-2}$ slope overlaid (dotted).

Per-grid-point detail (guaranteed budget), the specific points where error exceeds $\delta = 0.05$:

- `kabc-sg` exceeds δ at: Δ=1 (97%), Δ=1.5 (84%), Δ=2 (45%), N=30 (16%), d=50 (100%), d=100 (100%).
- `acb` exceeds δ at: Δ=1 (12%), Δ=1.5 (11%), Δ=2 (13%), N=120 (7%), d=50 (45%), d=100 (39%).
- `kabc` and `unif-kkm` exceed δ: nowhere.



(a) τ vs. ambient dimension d.



(b) τ vs. number of arms N.

Figure 2: Budget scaling along the two secondary sweep dimensions.

4.2 Restricted budget (budget_multiplier $= 0.25$): graceful degradation?

Table 1: Mean ARI at 0.25× budget, by Δ.

Δ	1.0	1.5	2.0	3.0	4.0	6.0
`kabc`	1.000	1.000	1.000	1.000	1.0	1.0
`kabc-sg`	0.880	0.917	0.960	0.996	1.0	1.0
`acb`	0.991	0.989	0.987	0.999	1.0	1.0
`unif-kkm`	1.000	1.000	1.000	1.000	1.0	1.0

Table 2: Mean ARI at 0.25× budget, by d and by N.

d	6	10	50	100
`kabc`	1.000	1.000	1.000	1.000
`kabc-sg`	1.000	0.996	0.385	0.026
`acb`	1.000	0.999	0.944	0.930
`unif-kkm`	1.000	1.000	1.000	1.000

N	30	60	120
`kabc`	1.000	1.000	1.000
`kabc-sg`	0.943	0.996	1.000
`acb`	0.999	0.999	0.998
`unif-kkm`	1.000	1.000	1.000

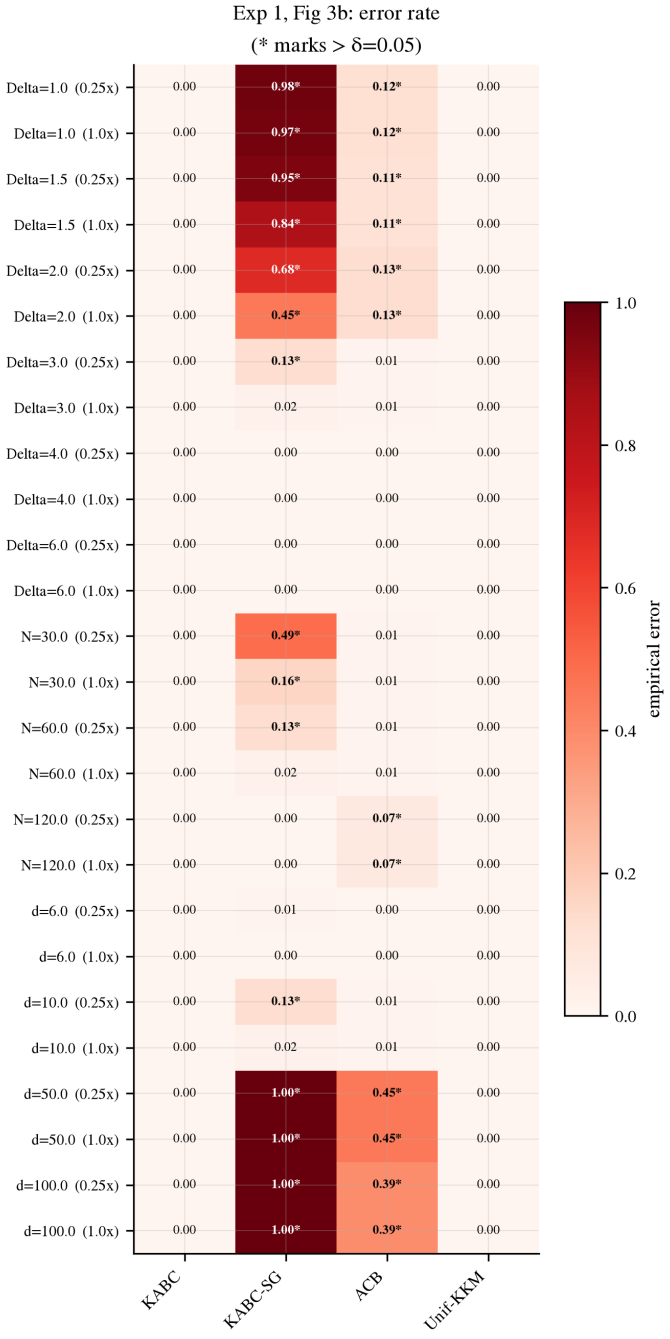


Figure 3: Error rate at every grid point × algorithm × budget regime. Asterisks mark cells exceeding $\delta = 0.05$.

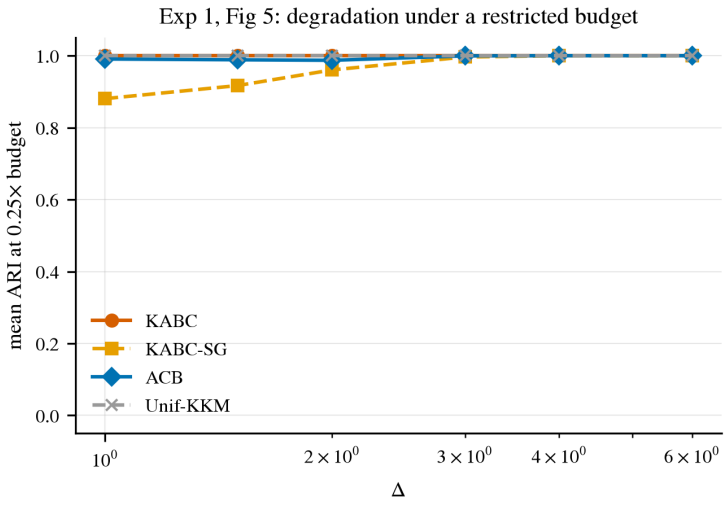
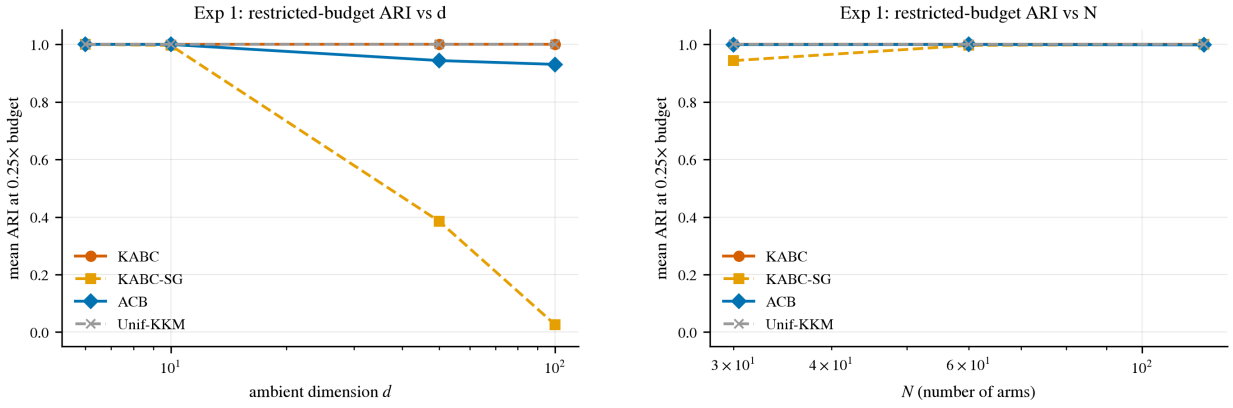


Figure 4: Mean ARI at 0.25× budget vs. Δ.



(a) Mean ARI at 0.25× budget vs. d.

(b) Mean ARI at 0.25× budget vs. N.

Figure 5: Restricted-budget degradation along the two secondary sweep dimensions.

5 Results — Experiment 3

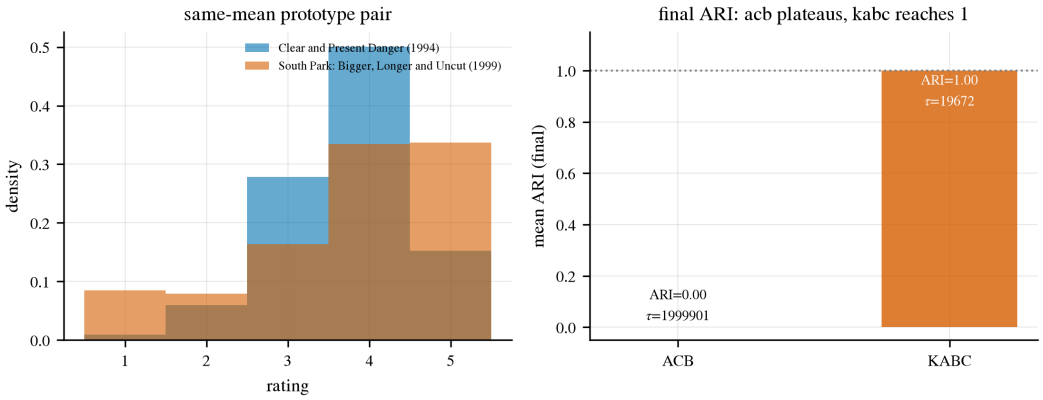


Figure 6: Left: rating histograms of the real same-mean movie pair. Right: final ARI per algorithm, annotated with mean τ.

Table 3: Main instance ($N = 40$, $K = 4$ prototypes), 100 reps/algorithm.

Algorithm	Stopped rate	Mean τ	Error rate	Mean ARI
`kabc`	100.0%	19,672	0.0%	1.000
`kabc-sg`	71.0%	69,326	63.0%	0.861
`acb`	0.0%	1,999,901	100.0%	0.000
`unif-kkm`	100.0%	2,000,000	0.0%	1.000

Table 4: Reduced instance ($N = 6$): TaS-FW vs. KABC.

Algorithm	Stopped rate	Mean τ	Error rate	Mean ARI	Mean wall-clock/rep
`tas-fw`	100.0%	4,357	0.0%	1.000	39.64 s
`kabc`	98.0%	1,852	2.0%	0.995	0.23 s

6 Inferences and Conclusions

6.1 KABC's δ-PAC guarantee holds empirically, everywhere it was tested. Across all 26 Experiment 1 grid points (both budget regimes) and both Experiment 3 instances, `kabc`'s empirical error was 0.0% except a single 2% figure on Experiment 3's tiny $N = 6$ reduced instance (still comfortably under $\delta = 0.05$). No other adaptive method in this benchmark achieved that consistency. This is the paper's central claim, holding up under a real, non-smoke-scale run across a genuinely varied grid and real MovieLens data, not just one convenient configuration.

6.2 KABC's budget scales with separation roughly as the theory predicts. Figure 1's dashed s_*^{-2} reference line and `kabc`'s actual curve track each other reasonably closely across the full Δ sweep, supporting Theorem 3.1's scaling claim empirically.

6.3 The variance-aware threshold is materially more robust than the kernel-uniform sub-Gaussian ablation — direct empirical evidence for the paper's own claimed benefit. `kabc-sg` is the clear failure mode of this benchmark: 34.5% aggregate error under the guaranteed budget (vs. `kabc`'s 0.0%), and a catastrophic collapse to near-random clustering (mean ARI 0.026) at $d = 100$ under a restricted budget. The mechanism, established by direct inspection during calibration: `kabc-sg`'s threshold uses one identical cutoff for every arm pair, so whichever single pair is noisiest for a given draw determines success or failure for the *whole* clustering; `kabc`'s variance-aware threshold sizes each pair's own cutoff from that pair's empirical spread, absorbing exactly this kind of pair-specific noise. Every `kabc-sg` failure inspected was a near-miss (one or two "straggler" arms failing to join their true cluster, ARI 0.86–0.98 typically) that ran out of budget rather than resolving — consistent with a uniformly slightly-too-tight threshold rather than a qualitative break.

6.4 ACB, on its own home turf, does not reliably satisfy its own confidence level at harder configurations — a calibration-generalization gap, not a design flaw. `acb`'s aggregate error (10.1%) exceeds $\delta = 0.05$, concentrated at small Δ (11–13%) and large d (39–45%). Unlike `kabc-sg`, `acb`'s stopped rate is 100% everywhere — its errors are confident, completed wrong answers, not budget-exhaustion near-misses. Given `acb`'s calibration-set/held-out error (2.0%/1.0%) looked fine in isolation, this is best read as the single fixed calibration instance failing to surface the harder Experiment 1 grid points, not as evidence against ACB's design. This should *not* be conflated with the mean-blindness story below, a qualitatively different and more severe failure mode.

6.5 On real data, ACB is not just imprecise — it is completely blind to same-mean, different-shape structure, exactly as the theory predicts. Experiment 3's headline number is unambiguous: `acb`'s mean ARI on the real same-mean movie pair is *exactly* 0.000, with 0% of runs ever stopping (it exhausts its full 2,000,000-pull budget every repetition). This is qualitatively different from the calibration gap in 6.4: a mean-based method has no signal whatsoever when two clusters share a mean, by construction, regardless of budget or calibration. `kabc` recovers the partition perfectly (ARI = 1.000) using under 1% of `acb`'s budget (19,672 vs. 1,999,901+ pulls). This directly confirms, on real data, the paper's core motivating claim: mean-based active clustering methods are blind to shape-only cluster differences that occur in practice.

6.6 Restricted-budget degradation is graceful for KABC (absent, in fact) and ACB, but sharp and dimension-driven for KABC-SG. Under a 4× budget cut, `kabc`'s mean ARI stays at exactly 1.000 across every one of the 13 grid points — no measurable degradation. `acb` degrades mildly and smoothly (0.93–0.99 in the worst cases). `kabc-sg` is the outlier: fine on the Δ and N sweeps (≥ 0.88 and ≥ 0.94 respectively) but collapsing specifically as ambient dimension grows (0.385 at $d = 50$, 0.026 at $d = 100$ — indistinguishable from random). This dimension-specific collapse (not mirrored in the other sweeps) suggests `kabc-sg`'s fixed, non-adaptive threshold interacts badly with higher-dimensional Gram-matrix statistics specifically, compounding the pair-uniformity fragility of §6.3.

6.7 TaS-FW is a fair, correctness-competitive baseline at the one scale its assumptions hold, at a real, quantified compute cost. On Experiment 3's $N = 6$ reduced instance, both `tas-fw` and `kabc` achieve essentially perfect recovery (ARI 1.000 vs. 0.995) with comparable, modest budgets (4,357 vs. 1,852 pulls). The real divergence is wall-clock cost per repetition: `tas-fw`

averages 39.64 s, `kabc` averages 0.23 s — roughly 172× more compute per decision, a real, measured confirmation of the design doc's "orders of magnitude more compute" characterization.

6.8 Open caveats for the write-up.

- `kabc-sg`'s and `acb`'s calibrated constants do not fully generalize from the single fixed calibration instance to Experiment 1's full real grid (§3, §6.4). Any claim about their error rates should be scoped to "as calibrated in this repo," not read as an indictment of the underlying algorithms' theoretical guarantees.
- Experiment 2 (identical means, different shapes — the synthetic complement to Experiment 3) remains a structural draft pending finalization of its instance families and grid; it is not included in this report.
- Figure 4 of the original design (appendix: theoretical vs. calibrated constants panel) requires a second, uncalibrated ($c = 1$) comparison run that has not been generated; not included here.

References

[1] Thuot, Vogt, Ghoshdastidar, Verzelen. *Kernel-based Active Clustering of Bandit Arms*. arXiv:2601.07535, 2026.

[2] Thuot, Carpentier, Giraud, Verzelen. *Active clustering with bandit feedback*. arXiv:2406.11485, 2024.

[3] Yavas, Huang, Tan, Scarlett. *A General Framework for Clustering and Distribution Matching with Bandit Feedback*. arXiv:2409.05072, 2024.